

PROJECT DOCUMENTATION

End-to-End DevSecOps CI/CD Pipeline on AWS

Automated delivery • Integrated security • Infrastructure as Code • Kubernetes • Observability

GITHUB → CI/CD → SCAN → ECR → EKS → MONITOR

Document type	Technical project guide
Cloud platform	Amazon Web Services (AWS)
Primary focus	Secure, repeatable application delivery

Prepared as a hands-on learning and portfolio project

Contents

- 01 Executive Summary
- 02 Project Objectives and Scope
- 03 Tools and Responsibilities
- 04 Architecture and End-to-End Workflow
- 05 Step-by-Step Implementation
- 06 Pipeline Quality and Security Gates
- 07 Suggested Repository Structure
- 08 Security and Operational Best Practices
- 09 Validation and Demo Checklist
- 10 Troubleshooting Guide
- 11 Interview Explanation
- 12 LinkedIn Post Draft
- 13 Cost, Cleanup, and Project Boundaries

01 Executive Summary

This project demonstrates an end-to-end DevSecOps workflow on AWS. Application code is managed in GitHub. A CI/CD orchestrator—Jenkins or GitHub Actions—automates testing, code-quality analysis, container building, vulnerability scanning, image publishing, and deployment. Terraform manages cloud infrastructure, Amazon EKS runs the containerized workload, and Prometheus with Grafana provides monitoring and visibility.

Project outcome A repeatable delivery path that makes releases traceable and introduces quality and security checks before deployment.

Project objectives

- Automate build, test, scan, publish, and deploy activities.
- Introduce code-quality and vulnerability checks into the delivery process.
- Provision infrastructure consistently using Terraform.
- Deploy a containerized application to Kubernetes on Amazon EKS.
- Observe application and cluster health using metrics and dashboards.
- Document deployment, validation, failure handling, and cleanup.

02 Project Objectives and Scope

The recommended first release uses one CI/CD orchestrator, one small web application, one AWS environment, and a single Kubernetes deployment. Keep the design small enough to understand and demonstrate end to end.

In scope	Optional extensions
GitHub repository and branch checks	Multi-environment promotion (dev/stage/prod)
CI pipeline with tests and scans	GitOps deployment with Argo CD or Flux
Docker image stored in ECR	Image signing and software bill of materials
Terraform-managed AWS resources	Private-only cluster and advanced network controls
Application on EKS	Progressive delivery / canary rollout
Prometheus and Grafana dashboards	Centralized logs and distributed tracing

03 Tools and Responsibilities

Technology	Responsibility	Project evidence
Git & GitHub	Version control, pull requests, source history	Repository and reviewed change
Jenkins or GitHub Actions	Orchestrate pipeline stages	Pipeline run and stage logs
SonarQube	Static analysis and quality gate	Quality-gate result
Docker	Package application and runtime dependencies	Image build
Trivy	Scan dependencies, filesystem, and image	Scan report and policy result

Amazon ECR	Private registry for versioned images	Image tag or digest
Terraform	Provision and manage infrastructure	Plan and apply output
Amazon EKS	Managed Kubernetes control plane and workload platform	Successful rollout
Prometheus	Collect time-series metrics	Targets and metrics
Grafana	Dashboards and alert visualization	Dashboard screenshot

04 Architecture and End-to-End Workflow

Logical flow Developer → GitHub → CI pipeline → Tests / SonarQube → Docker build → Trivy scan → Amazon ECR → Amazon EKS → Prometheus / Grafana

Terraform is used to create and update the supporting AWS infrastructure. It is normally run as a controlled infrastructure workflow with reviewed plans, rather than blindly applying infrastructure changes on every application commit.

1. A developer commits code and opens or merges a pull request.
2. GitHub triggers the selected CI/CD workflow.
3. The workflow checks out code, installs dependencies, and runs automated tests.
4. SonarQube evaluates code quality against the configured quality gate.
5. Docker builds a versioned application image.
6. Trivy scans the source/dependencies and/or built image according to the chosen policy.
7. If required gates pass, the workflow authenticates to ECR and pushes the image.
8. The deployment stage updates the Kubernetes Deployment to the exact image tag or digest.
9. The workflow checks rollout status and runs a smoke test.
10. Prometheus collects metrics; Grafana displays dashboards and alerts.

05 Step-by-Step Implementation

Step 1 — Build a small application

Choose a simple web application with automated tests and a health endpoint such as /health. Keep the application small; the learning goal is the delivery pipeline.

Step 2 — Create the GitHub repository

Add application source, tests, Dockerfile, .dockerignore, Kubernetes manifests, Terraform configuration, monitoring configuration, and README. Use pull requests and protect the main branch.

Step 3 — Prepare AWS access and cost controls

Use a dedicated project environment. Create least-privilege identities and set a budget or billing alert. Prefer short-lived credentials and avoid storing AWS access keys in source control.

Step 4 — Provision infrastructure with Terraform

Define the required VPC/subnets, IAM roles, ECR repository, EKS cluster, and worker capacity. Run terraform fmt and validate, inspect terraform plan, and apply only after review. Protect the state backend and restrict access to plan artifacts.

Step 5 — Configure the CI/CD orchestrator

Choose Jenkins or GitHub Actions. For GitHub Actions, prefer OIDC role assumption for AWS access. For Jenkins, secure the controller and agents, manage credentials centrally, and grant the pipeline only the permissions it needs.

Step 6 — Add tests and SonarQube

Run unit tests and publish coverage where available. Configure the SonarQube scanner and project quality gate. Make the pipeline fail when required quality conditions are not met.

Step 7 — Build the Docker image

Use a small trusted base image, pin versions where practical, add a `.dockerignore` file, run as a non-root user where feasible, and do not bake secrets into image layers.

Step 8 — Scan with Trivy

Scan dependencies/filesystem and the final container image. Decide which severities block promotion, how exceptions are approved, and when exceptions expire. Save a concise scan summary as pipeline evidence.

Step 9 — Publish to Amazon ECR

Authenticate from the pipeline using the approved identity. Tag the image with the Git commit SHA or release version, push it to ECR, and record the tag or digest used for deployment.

Step 10 — Deploy to Amazon EKS

Configure Kubernetes access with least privilege. Update the Deployment to the exact image version. Configure readiness and liveness probes, resource requests and limits, rolling-update behavior, and a Service.

Step 11 — Validate the deployment

Wait for rollout completion, check pod status and events, and run a smoke test against the application. If validation fails, collect logs and roll back to a known-good image.

Step 12 — Add Prometheus and Grafana

Install a maintained monitoring stack using an appropriate chart or operator. Monitor CPU, memory, pod restarts, request rate, errors, latency, and deployment health. Add useful alerts and test them.

Step 13 — Capture portfolio evidence

Capture sanitized screenshots of the architecture, pipeline run, SonarQube gate, Trivy report, ECR image, EKS rollout, and Grafana dashboard. Never publish credentials, tokens, private keys, or sensitive account details.

06 Pipeline Quality and Security Gates

Stage	Example pass condition	Failure response
Unit tests	All required tests pass	Stop pipeline; show test report
SonarQube	Configured quality gate passes	Block promotion until fixed or approved exception
Docker build	Image builds successfully	Stop and report build logs

Trivy	No unapproved findings above policy threshold	Block publish/deploy or require documented exception
ECR publish	Image pushed with traceable version	Stop deployment; check auth/tag/permissions
EKS rollout	Deployment becomes available	Collect events/logs; roll back if appropriate
Smoke test	Health endpoint returns expected response	Fail release and investigate
Monitoring	Metrics visible and alerts tested	Fix scrape, permissions, or dashboard configuration

Policy tip Write down your vulnerability threshold and exception process. A scan is only an effective gate when the team has agreed what its result means.

07 Suggested Repository Structure

```

devsecops-aws-pipeline/
├── app/                    # application source and tests
├── Dockerfile
├── .dockerignore
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml      # optional
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── modules/          # optional
├── .github/workflows/     # GitHub Actions, if selected
├── Jenkinsfile            # Jenkins, if selected
├── monitoring/           # dashboards and configuration
└── README.md

```

08 Security and Operational Best Practices

- Apply least privilege to AWS IAM and Kubernetes RBAC; separate infrastructure and application deployment permissions where practical.
- Prefer OIDC or short-lived credentials; never commit access keys, tokens, kubeconfig files, or secret values.
- Protect branches and require reviews. Pin trusted third-party CI actions/plugins to known versions.
- Keep container images minimal, patch base images, and avoid running as root where feasible.
- Scan before promotion and document accepted vulnerabilities, owners, and expiry dates.
- Protect Terraform state and restrict access to plans, logs, and artifacts that may contain sensitive values.
- Restrict network exposure, use TLS for public endpoints, and review security groups and cluster access.
- Set budgets, logging/retention controls, and a cleanup plan for non-production resources.

09 Validation and Demo Checklist

- ☐ A code change triggers the expected workflow.
- ☐ A failing unit test stops the pipeline.

- ☐ A failed SonarQube gate blocks promotion.
- ☐ A Trivy finding is handled according to the documented policy.
- ☐ The published ECR image is traceable to a commit.
- ☐ The EKS Deployment runs the intended image version.
- ☐ The application health endpoint passes after deployment.
- ☐ A failed rollout can be diagnosed and rolled back.
- ☐ Prometheus has healthy scrape targets and Grafana displays current metrics.
- ☐ Screenshots and README contain no secrets or sensitive identifiers.

10 Troubleshooting Guide

Symptom	Things to check
CI cannot access AWS	OIDC trust policy, role permissions, region, audience/subject conditions
ECR push denied	Repository URI, login region, IAM permissions, image tag
EKS deployment cannot pull image	Node/pod permissions, ECR access, image URI and tag
Pods are not Ready	Readiness probe path/port, container logs, resource limits, events
SonarQube gate fails	Quality profile, new-code findings, coverage, scanner configuration
Trivy blocks the build	Finding severity, package/base image version, approved exception process
Prometheus has no data	ServiceMonitor/scrape config, labels, network access, target health
Grafana dashboard is empty	Data source, query, time range, metric names, permissions

11 Interview Explanation

60–90 second summary “I designed an end-to-end DevSecOps pipeline on AWS. Developers push code to GitHub, which triggers Jenkins or GitHub Actions. The pipeline runs tests and SonarQube analysis, builds a Docker image, and scans it with Trivy. When the required gates pass, the image is tagged and pushed to Amazon ECR. Terraform manages the AWS infrastructure, and the application is deployed to Amazon EKS with rollout and health checks. Prometheus and Grafana provide visibility into application and cluster health. The goal is to make releases repeatable, traceable, and security-focused.”

Questions to prepare for

- Why did you choose Jenkins or GitHub Actions?
- How does the pipeline authenticate to AWS without exposing long-lived credentials?
- What happens when SonarQube or Trivy fails?
- How do you ensure the deployed image matches the tested image?
- How do you protect Terraform state?
- How do you roll back a failed Kubernetes deployment?
- Which metrics and alerts would you use to identify a service problem?

12 LinkedIn Post Draft

□ Project: End-to-End DevSecOps CI/CD Pipeline on AWS

I'm building a hands-on DevSecOps workflow that connects source control, CI/CD automation, container security, Kubernetes, and observability.

- Git & GitHub — Version control
- Jenkins / GitHub Actions — Pipeline automation
- SonarQube — Code quality analysis
- Docker + Trivy — Containerization and vulnerability scanning
- Amazon ECR — Container image registry
- Terraform — Infrastructure as Code
- Amazon EKS — Kubernetes deployment
- Prometheus + Grafana — Monitoring and dashboards

The goal is to automate testing, security checks, image publishing, deployment, and post-deployment validation—with traceable releases and monitoring.

Key takeaway: security and observability are most useful when they are part of the delivery workflow.

#AWS #DevSecOps #CICD #Jenkins #GitHubActions #Docker #Kubernetes #Terraform #AmazonEKS
#AmazonECR #SonarQube #Trivy #Prometheus #Grafana

13 Cost, Cleanup, and Project Boundaries

This document is an implementation blueprint, not evidence that AWS resources have already been deployed. EKS, compute, networking, load balancers, storage, and monitoring resources may incur charges. Check current AWS pricing and tool documentation before creating resources. Use a small environment, set budget alerts, and remove resources when the project is no longer needed. Review Terraform destroy plans carefully and preserve any data you need before cleanup.

Before publishing Only describe stages as completed if you have run them. Use real, sanitized screenshots from your own environment to support project claims.